

Milestone 1: Batching & Continuous Batching

CS349D — Spring 2026 Name: Hiva Mohammadzadeh

1. Overview

This report describes the design and implementation of batched decoding and continuous (iteration-level) batching in MiniEngine, a minimal LLM serving engine. The baseline scheduler processes one request at a time, prefill to completion, leaving the GPU severely underutilized. My optimized scheduler admits multiple concurrent requests and decodes them together in a single GPU forward pass, dramatically improving throughput at higher concurrency levels.

2. Hardware Configuration

- **VM:** Google Deep Learning VM (`cs349d-milestone1`, `us-central1-a`)
- **Image:** `common-cu129-ubuntu-2404-nvidia-580-stage`
- **GPU:** NVIDIA Tesla T4 (15 GB VRAM)
- **CPU:** 8 vCPUs
- **DRAM:** 50 GB

I chose this configuration because the Qwen3-0.6B model (~1.2 GB in float16) fits comfortably in the T4's 15 GB VRAM. However, KV caches at longer sequence lengths consume significant memory, I found that `max_running=16` caused OOM at `conc=16`, so I tuned down to `max_running=8` which fits within the T4's budget. The T4 is the most cost-effective GPU option for this milestone since the 0.6B model is small enough that compute not memory is the primary bottleneck during decode at lower batch sizes. The 50 GB DRAM is more than sufficient for tokenizer loading and request buffering. A larger GPU (e.g., L4 24GB) would allow higher `max_running` values or serving the full Qwen3-4B model, but is not necessary to demonstrate the batching gains at this scale.

3. Design & Implementation

I modified three files: `model.py`, `engine.py`, and `scheduler.py`.

3.1 Model Changes (`model.py`)

I threaded an optional `attention_mask` parameter through the entire model stack: `CausalLM` → `TransformerModel` → `TransformerBlock` → `Attention`. When an attention mask is provided, it is passed directly to `F.scaled_dot_product_attention(attn_mask=...)`, replacing the `is_causal` shortcut. When no mask is provided (unbatched prefill/decode), the original `is_causal` logic is preserved. This ensures correctness for both batched and unbatched paths with zero overhead for the unbatched case.

3.2 Batched Decode (`engine.py`)

I added a `batched_decode(requests)` method to the Engine class that processes multiple decode requests in a single forward pass:

1. **Stack input tokens:** Collect the last generated token from each request into a `(batch, 1)` tensor.
2. **Pad KV caches:** Each request has a KV cache of different length. I pad all caches to the maximum length in the batch along the sequence dimension using `F.pad`.
3. **Build attention mask:** Construct a `(batch, 1, 1, max_cache_len + 1)` mask that places `-inf` at padding positions between each request's actual cache end and the max length. This prevents the model from attending to padding tokens.
4. **Position IDs:** Each request's position is set to its own cache length (not the padded length), ensuring RoPE embeddings are correct, previously cached K/V already have the right RoPE from when they were computed.
5. **Batched forward pass:** Run a single model forward with the padded batch.

6. **Extract per-request KV:** The returned KV has shape (batch, kv_heads, max_cache_len+1, head_dim) with padding in the middle. For each request with actual cache length L, I keep positions [:L] (old) and [-1:] (new token), concatenating them to form the updated cache without padding.
7. **Sample per request:** Apply each request’s sampling parameters independently.

Prefill remains unbatched because different prompt lengths would require complex variable-length padding with causal masking. Since prefill is a one-time cost per request and decode is where throughput gains live, this is a good tradeoff.

3.3 Continuous Batching Scheduler (scheduler.py)

I rewrote `step()` to implement iteration-level batching with three phases:

- **Phase 1 — Admit & Prefill:** Under a lock, admit waiting requests up to `max_running` slots. Each new request is prefilled individually (variable prompt lengths). Requests that immediately hit a stop token are retired.
- **Phase 2 — Batched Decode:** All running requests (including those just prefilled) are decoded together in a single call to `batched_decode()`. This is the key throughput optimization.
- **Phase 3 — Retire:** Finished requests (stop token or max length) are removed and their KV caches freed. Their slots are immediately available for the next `step()` iteration.

The batch composition changes every iteration, a request that finishes frees its slot for a waiting request on the very next step, and a newly prefilled request joins the decode batch in the same step. No request has to wait for the entire batch to drain.

The `max_running` parameter controls the maximum batch size and thus GPU memory usage. After tuning, I settled on `max_running=8` for the T4 GPU to balance throughput and memory.

4. Baseline Results

Baseline configuration: Qwen/Qwen3-0.6B, naive FCFS scheduler (one request at a time, no batching), 100 requests, input length 1024, output length 1024.

Conc	TTFT p50 (ms)	TTFT p99 (ms)	Compl p50 (ms)	Compl p99 (ms)	TPOT p50 (ms)	TPOT p99 (ms)	GenTok/sOK	
1	99	106	103	32,318	31.2	31.7	32	100
2	197	32,347	32,070	64,256	31.2	31.4	32	100
4	31,063	79,746	60,393	92,587	29.4	30.7	34	100
8	91,345	152,436	91,691	182,880	29.6	30.1	34	100
16	148,529	227,943	178,618	335,495	115.9	148.4	34	100
32	190,384	329,188	275,617	534,510	108.8	363.3	14	76

Key observations:

- Throughput is flat at ~32-34 tok/s regardless of concurrency, the GPU processes one request at a time, so additional requests just queue up.
- TTFT and completion latency grow linearly with concurrency since requests wait in line.
- At concurrency 32, some requests time out (only 76/100 succeed).

5. Optimized Results

5.1 max_running=16

With `max_running=16`, the scheduler batches up to 16 requests simultaneously. This worked well up to `conc=8` but hit OOM at `conc=16` due to the T4’s 15 GB VRAM limit.

Conc	TTFT p50 (ms)	TTFT p99 (ms)	Compl p50 (ms)	Compl p99 (ms)	TPOT p50 (ms)	TPOT p99 (ms)	GenTok/sOK
1	101	118	36,084	37,116	35.5	36.2	28 100
2	132	211	44,001	45,101	43.2	44.0	46 100
4	238	346	65,410	65,889	63.8	64.3	62 100
8	351	863	120,891	123,822	120.3	120.8	66 100
16	OOM	—	—	—	—	—	— 9
32	OOM	—	—	—	—	—	— 0

5.2 max_running=8

To resolve OOM, I reduced `max_running` to 8 and set `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to reduce CUDA memory fragmentation. This caps the batch at 8 concurrent decodes, requests beyond 8 queue and are admitted as slots free up.

Conc	TTFT p50 (ms)	TTFT p99 (ms)	Compl p50 (ms)	Compl p99 (ms)	TPOT p50 (ms)	TPOT p99 (ms)	GenTok/sOK
1	101	116	36,174	37,181	35.5	36.2	28 100
2	132	211	43,731	44,702	43.0	43.6	46 100
4	237	346	65,316	65,766	63.7	64.1	62 100
8	353	865	120,735	123,995	120.0	120.8	67 100
16	119,786	123,623	124,000	246,752	118.0	121.1	67 100
32	123,397	244,582	320,159	512,163	254.5	359.3	66 100

6. Analysis

Throughput scaling. The baseline throughput is flat at ~32-34 tok/s regardless of concurrency, since only one request is decoded at a time and additional requests simply queue up. With batched decode, throughput scales with concurrency: 28 tok/s at conc=1, 46 tok/s at conc=2 (1.4x), 62 tok/s at conc=4 (1.8x), and plateaus at ~66-67 tok/s from conc=8 onward, a **2x improvement** over the baseline peak. The plateau at conc=8 makes sense: with `max_running=8`, the batch size is capped at 8 regardless of how many requests are queued, so conc=16 and 32 see the same decode throughput.

TTFT improvement. This is where continuous batching shows the most dramatic gains. In the baseline, requests at higher concurrency must wait for all preceding requests to fully complete before they even begin, leading to TTFT p50 of 31s at conc=4 and 148s at conc=16. With continuous batching, new requests are admitted and prefilled as soon as a slot opens, so TTFT stays low at lower concurrencies: 101ms at conc=1, 132ms at conc=2, and just 237ms at conc=4; a **130x improvement** over the baseline’s 31,063ms. At conc=16 and 32, TTFT rises to ~120s because all 8 slots are occupied and new requests must wait for a slot to free up, but this is still better than the baseline’s 148s and 190s respectively.

Completion latency and success rate. At conc=32, the baseline only completed 76/100 requests (24 timed out), while the optimized engine completed all 100. The optimized completion latency at conc=32 (320s p50) is comparable to the baseline (275s p50), but with 100% success rate versus 76%. At conc=16, optimized completion p50 is 124s vs the baseline’s 178s — a 30% reduction.

Concurrency=1 overhead. At conc=1, the optimized engine (28 tok/s) is slightly slower than the baseline (32 tok/s). This is expected, the batched decode path has overhead from KV cache padding, attention mask construction, and per-request KV extraction that provides no benefit with a single request. The overhead is small (~12%) and is quickly offset by the throughput gains at higher concurrency.

Memory constraints. I initially set `max_running=16` but hit OOM on the T4 at conc=16. Reducing to `max_running=8` with `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` resolved the issue. This highlights a key tradeoff in LLM serving: larger batches improve GPU compute utilization but require

proportionally more memory for KV caches. With the 0.6B model, each request’s KV cache for 2048 tokens is roughly ~150 MB, so 8 concurrent caches consume ~1.2 GB on top of the model’s ~1.2 GB footprint.

7. Conclusion

Batched decoding and continuous batching yielded significant improvements across all key metrics. Peak throughput increased from 34 tok/s to 67 tok/s (2x), TTFT at conc=4 dropped from 31s to 237ms (130x), and at conc=32 the optimized engine completed 100/100 requests versus the baseline’s 76/100. The main tradeoff is memory: larger batch sizes improve compute utilization but require more VRAM for KV caches, which I navigated by tuning `max_running` from 16 down to 8 on the T4. These results confirm that even simple batching and continuous scheduling, without any memory management optimizations like paging can substantially improve LLM serving performance.